



Sure Trust- SKILL UPGRADATION FOR RURAL YOUTH EMPOWERMENT
Puttaparthi, Andhra Pradesh

Project Report On
Art Challenge App

By
Sayan Mondal
(Sayan G30-FSD)

Under The Guidance Of
Akshara A
Trainer of Internship

Certificate

This is to certify that the project entitled "Art Challenge App " has been successfully completed by Sayan Mondal (Sayan G30-FSD) as a part of the Project requirement under the SURE Trust Skill Upgradation Program.

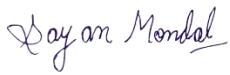
The work presented in this report is original and has not been submitted to any other institution or university for the award of any degree or diploma.

Ms. Akshara A Project Mentor / Trainer	 Sayan Student / Developer
--	--

Declaration

I, Sayan, hereby declare that this project report entitled "Art Challenge App" is a Bonafide record of work carried out by me under the supervision of Ms. Akshara A, Trainer, SURE Trust, Puttaparthi.

I declare that this project has been prepared by me and has not been submitted to any other institution for the award of any degree, diploma, or fellowship. All the information furnished in this report is true and original to the best of my knowledge.

Signature: 

Name:- Sayan

ID:- Sayan G30-FSD

Acknowledgement

I would like to express my sincere gratitude to all those who have contributed towards the successful completion of this project.

First and foremost, I am deeply thankful to Ms. Akshara A, my Project Mentor and Trainer at SURE Trust, for her invaluable guidance, continuous encouragement, and constructive feedback throughout the development of this project. Her expertise and insights have been instrumental in shaping this work.

I extend my heartfelt thanks to the management and faculty of SURE Trust, Puttaparthi, for providing an excellent learning environment and the resources required to complete this project successfully.

I am also grateful to my peers and batchmates for their cooperation, discussions, and moral support, which helped me stay motivated through the challenges faced during this project.

Finally, I owe a debt of gratitude to my family for their unwavering support, patience, and encouragement throughout my academic journey.

Sayan

1. Executive Summary

Art Challenge App is a real-time, browser-based application that enables users to draw in mid-air using hand gestures captured via a webcam. Inspired by spatial computing paradigms seen in science-fiction interfaces, the system leverages Google's MediaPipe Hands AI model for skeleton tracking and translates detected gestures into creative drawing actions on an HTML5 Canvas.

The application supports a two-hand workflow: one hand is dedicated to drawing and erasing, while the other performs non-destructive transformations (move, scale, rotate) on existing strokes — all without touching a keyboard or mouse.

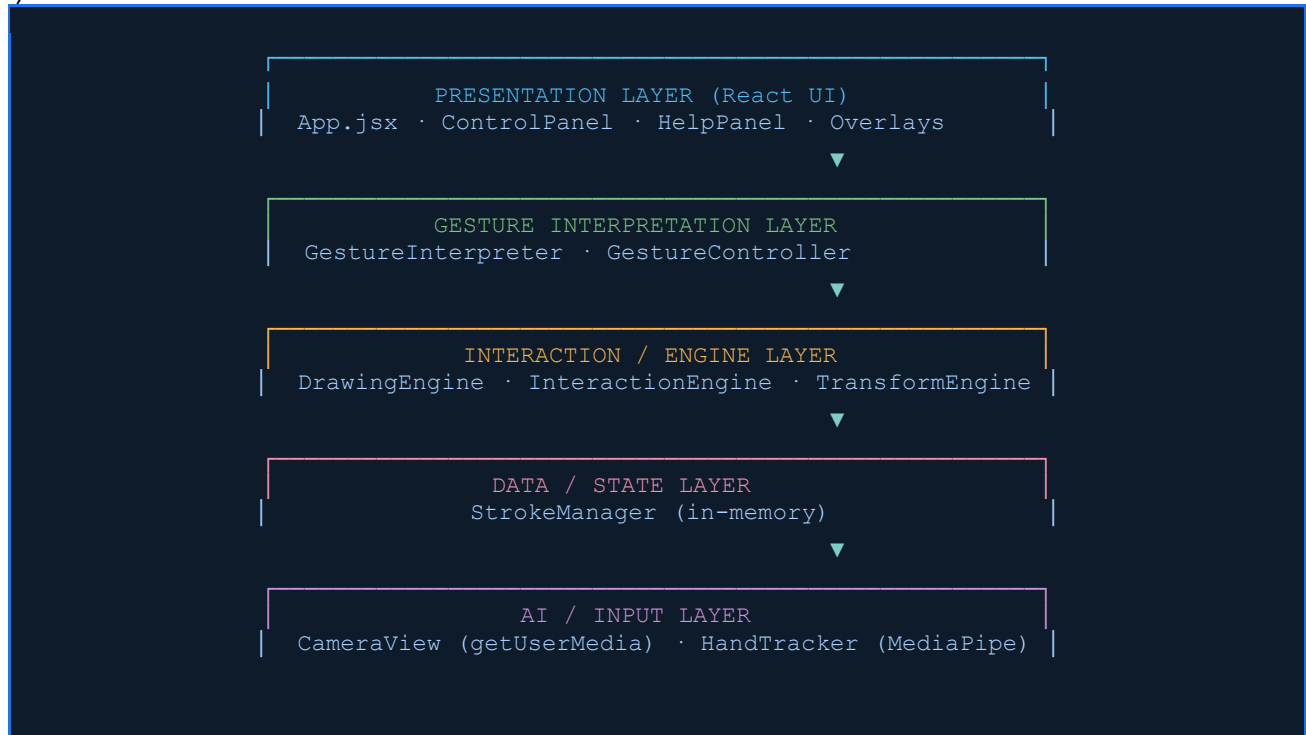
Attribute	Details
Project Name	Art Challenge App (AirDrawer)
Type	Frontend Web Application (SPA)
Core Technology	React 19 + Vite 8 + MediaPipe Hands + Three.js
Hand Tracking	MediaPipe Hands (up to 2 hands, 21 landmarks each)
Rendering	HTML5 Canvas 2D API with WebGL via Three.js
Target Performance	60 FPS smooth interaction
Developer	Sayan Mondal

2. System Architecture

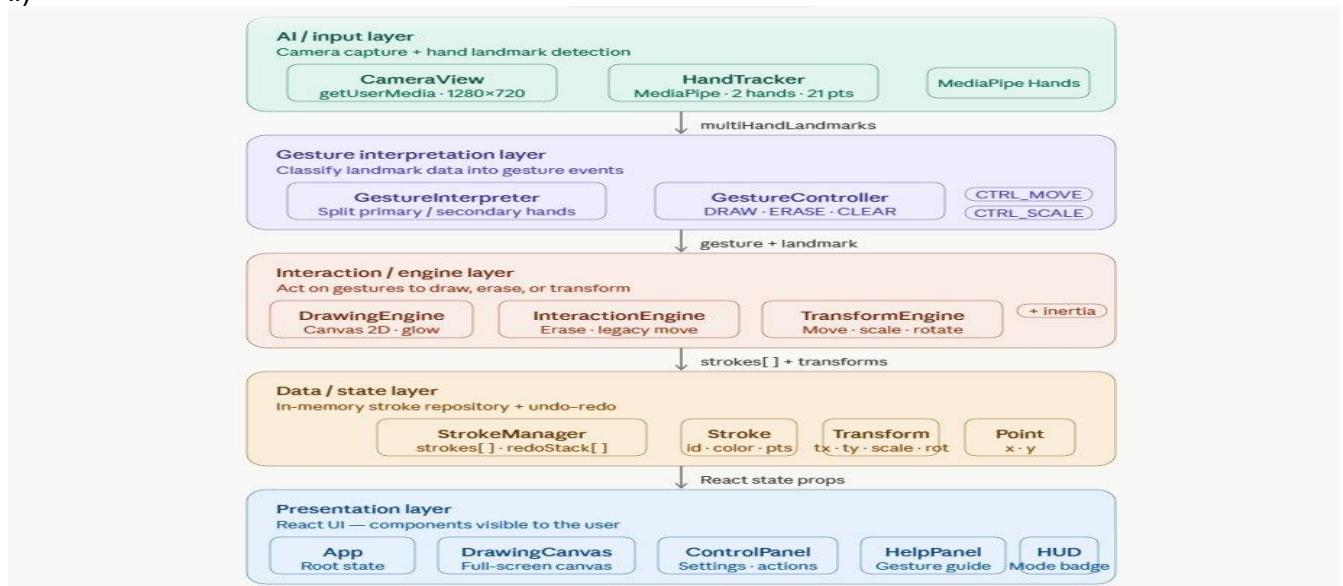
2.1 High-Level Overview

The application follows a layered, event-driven architecture. Camera frames flow into the AI tracking layer, producing hand landmark data that feeds the gesture interpretation layer. Gesture events then propagate to the interaction and rendering layers, completing a real-time feedback loop at ~60 frames per second.

i)



ii)



2.2 Module Breakdown

Module / File	Layer	Responsibility
CameraView.jsx	AI / Input	Initialises webcam via getUserMedia; feeds frames to HandTracker at 60 FPS
handTracking.js	AI / Input	Wraps MediaPipe Hands; detects up to 2 hands and returns 21 landmarks each
gestureController.js	Gesture	Maps landmark geometry to drawing gestures: DRAW, ERASE, CLEAR, MOVE, IDLE
gestureInterpreter.js	Gesture	Splits two-hand input into primary (draw) and secondary (control) streams with gesture smoothing (4-frame debounce)
strokeManager.js	Data	In-memory stroke store with undo/redo stacks; spatial queries for nearest/intersecting strokes
drawingEngine.js	Rendering	Canvas 2D renderer; draws all strokes with glow effects; renders selection guides (move crosshair, scale rings, rotation arc)
interactionEngine.js	Interaction	Handles point-erase and legacy move; hit-tests strokes within a configurable radius
transformEngine.js	Interaction	Non-destructive move/scale/rotate via per-stroke transform matrix; post-release inertia and 45° rotation snap
DrawingCanvas.jsx	Presentation	Full-screen canvas host; bridges React state with engine instances; exposes clear/undo/redo/save via ref
ControlPanel.jsx	Presentation	Glassmorphism settings sidebar: colour palette, brush sliders, action buttons, social links
HelpPanel.jsx	Presentation	Modal gesture cheat-sheet; animated with Framer Motion spring transitions
App.jsx	Root	Application root; owns global state (gesture, settings, UI toggles); orchestrates all components

2.3 Data Flow Diagram

The following sequence describes a single draw stroke from hand movement to rendered pixel:

- Webcam captures a video frame (CameraView).
- Frame is sent to MediaPipe Hands model (HandTracker.send).
- MediaPipe returns multiHandLandmarks with 21 (x,y,z) points per hand.
- GestureInterpreter splits landmarks into primary/secondary hand streams.
- GestureController detects DRAW gesture on primary hand (index finger up).
- App.jsx sets gesture state → triggers useEffect in DrawingCanvas.
- DrawingCanvas maps landmark coordinates to canvas pixels (with horizontal mirror flip).
- A smoothed point is appended to the current in-progress path (0.85 exponential smoothing).
- On gesture change away from DRAW, the path is committed to StrokeManager.
- DrawingEngine's 60 FPS rAF loop reads all strokes and redraws the canvas.

3. Component Architecture

3.1 React Component Tree

```
App (root state: settings, gesture, controlGesture, camera, ui)
├── CameraView          ← webcam + MediaPipe frames
├── DrawingCanvas (ref)  ← canvas + all engines
│   ├── DrawingEngine
│   ├── StrokeManager   ← stroke store + undo stack
│   ├── InteractionEngine
│   └── TransformEngine ← move/scale/rotate + inertia
├── ControlPanel        ← settings sidebar
├── HelpPanel           ← gesture modal
├── Fingertip Overlays (primary) ← inline JSX divs
├── Fingertip Overlays (control) ← inline JSX divs
└── Gesture Status HUD  ← Framer Motion AnimatePresence
```


3.2 State Management

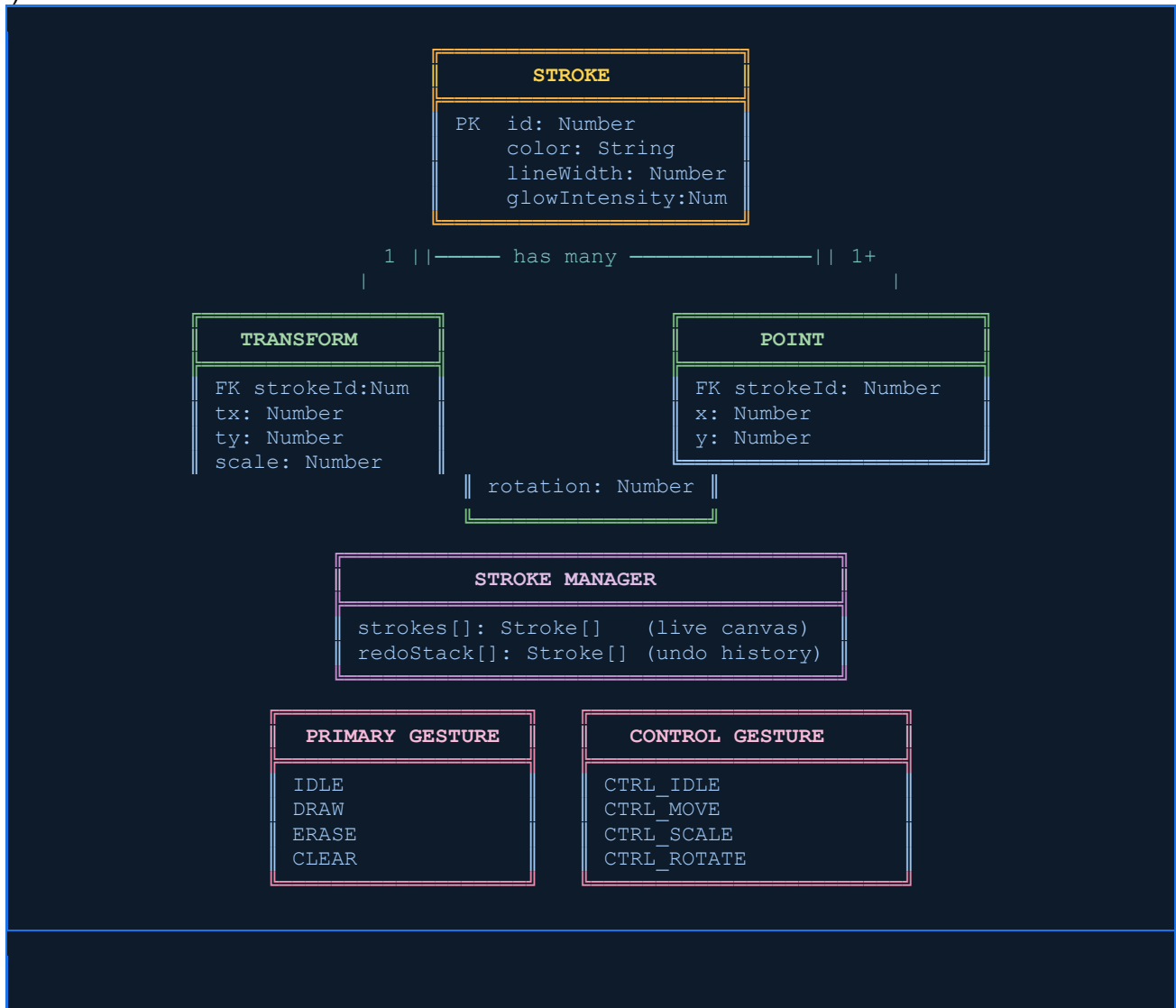
The application uses React's built-in `useState` and `useRef` hooks exclusively — no external state library is required. The table below maps each state variable to its owner and consumers.

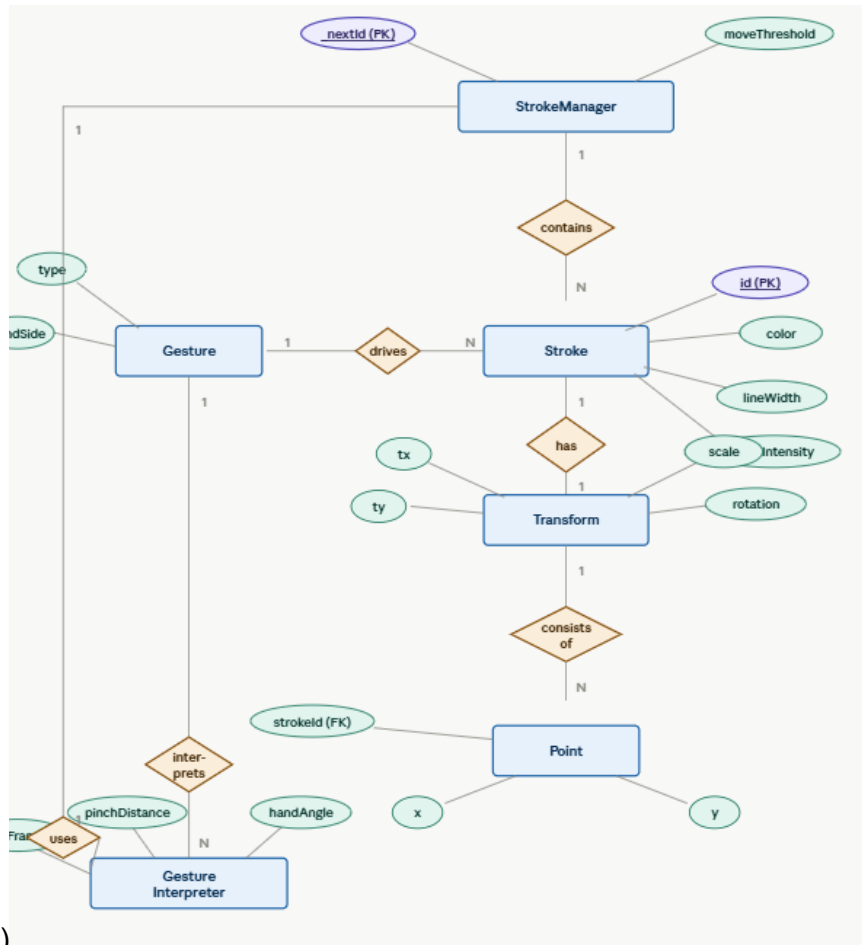
State Variable	Type	Owner	Consumer
settings	Object	App	DrawingCanvas, ControlPanel
gesture	String enum	App	DrawingCanvas, HUD overlay
landmark	Object {x,y,z}	App	DrawingCanvas, fingertip divs
fingertips[5]	Array	App	Fingertip overlay divs
controlGesture	String enum	App	DrawingCanvas, HUD overlay
controlLandmark	Object {x,y,z}	App	DrawingCanvas
controlPinchDelta	Number	App	DrawingCanvas → TransformEngine
controlAngleDelta	Number	App	DrawingCanvas → TransformEngine
cameraVisible	Boolean	App	CameraView (conditional render)
gesturesEnabled	Boolean	App	onResults callback guard
isHelpOpen	Boolean	App	HelpPanel

4. Entity Relationship Diagram (ERD)

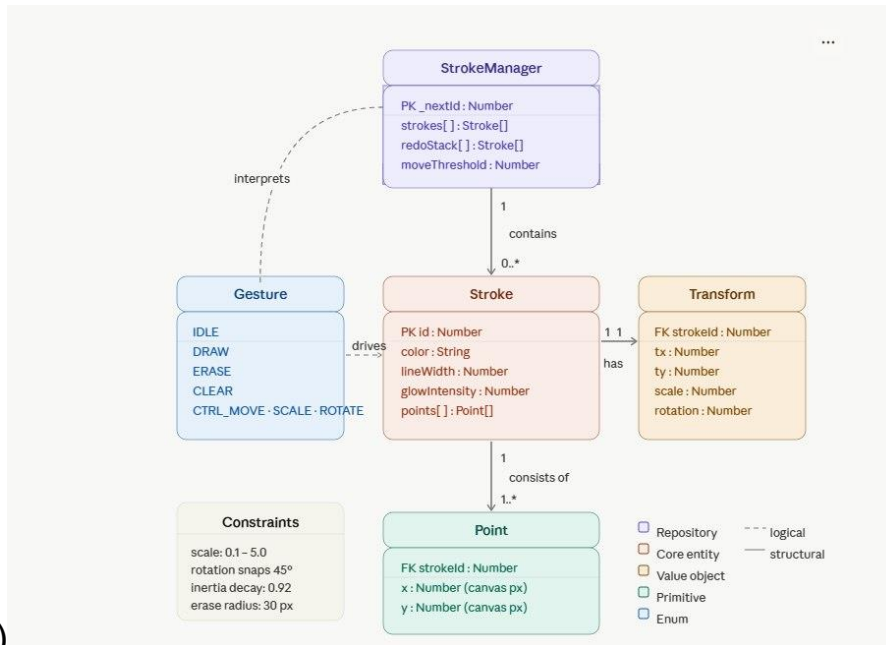
Although this is a frontend-only, in-memory application (no database), the domain objects can be modelled as entities with clear relationships. The ERD below describes the logical data model used at runtime.

i)





ii)



iii)

4.1 Entity Descriptions

5.1 MediaPipe Hand Landmarks

MediaPipe Hands returns 21 3-D landmarks per detected hand. The key landmarks used by this application are:

Index	Joint	Usage
0	Wrist	Rotation axis origin; hand angle computation
4	Thumb tip	Pinch distance calculation (erase / scale)
8	Index tip	Primary drawing cursor position
9	Middle base (MCP)	Hand orientation angle
12	Middle tip	Second-finger-up test (MOVE / two-finger)
16	Ring tip	Down-finger test (all-up = ROTATE)
20	Pinky tip	Down-finger test

5.2 Gesture Decision Logic (Primary Hand)

Gesture	Finger Condition	Action
ERASE	Thumb–Index pinch distance < 0.05 (normalised)	Remove intersecting strokes
CLEAR	All fingers closed (fist)	Clear entire canvas
DRAW	Index up only	Append smoothed point to current path
IDLE	Any other combination	Commit in-progress stroke; pause drawing

5.3 Gesture Decision Logic (Control Hand)

Gesture	Finger Condition	Action
CTRL_SCALE	Pinch distance < 0.06	Scale selected stroke by pinch delta × 8
CTRL_ROTATE	All 4 fingers up (open palm)	Rotate by wrist angle delta; snap 45° on release
CTRL_MOVE	Index + Middle up only	Translate selected stroke by cursor delta
CTRL_IDLE	Any other (after 4-frame debounce)	Release selection; apply inertia if moving

6. Non-Destructive Transform Engine

A key design decision is that stroke points are never mutated during transform operations. Instead, each stroke carries a Transform object. At render time, `TransformEngine.getTransformedPoints(stroke)` applies the following pipeline to each original point:

- Compute centroid (cx, cy) from all original points.
- Translate each point to origin: (x - cx, y - cy).
- Apply scale: multiply x and y by `stroke.transform.scale`.
- Apply rotation: standard 2D rotation matrix using cos/sin of `stroke.transform.rotation`.
- Translate back and apply offset: add centroid + (tx, ty).

This pipeline ensures that scaling and rotation always occur relative to the stroke's own centroid, not a global origin. Benefits include: instant undo of all transforms (reset tx=0, ty=0, scale=1, rotation=0); smooth inertial drift after releasing a move gesture; and 45° rotation snap on gesture release for precision alignment.

6.1 Inertia System

When the CTRL_MOVE gesture is released, if the trailing velocity exceeds 0.5px/frame, the TransformEngine begins a `requestAnimationFrame` inertia loop. Each tick decays both velocity components by a factor of 0.92 until the magnitude drops below 0.1px/frame, at which point animation stops. This provides a natural "flick" feel.

7. Technology Stack

Technology	Version	Purpose
React	^19.2.4	UI component tree, state management via hooks
Vite	^8.0.1	Lightning-fast dev server with HMR; production bundler
MediaPipe Hands	^0.4.1675469240	Real-time AI hand landmark detection (via CDN)
Three.js	^0.183.2	WebGL context for hardware-accelerated rendering
Framer Motion	^12.38.0	Spring animations for UI panels and gesture HUD
Lucide React	^1.7.0	Icon set for ControlPanel buttons
HTML5 Canvas 2D	Browser API	Primary drawing surface with glow/shadow effects
getUserMedia API	Browser API	Webcam stream acquisition at up to 1280×720
CSS Glassmorphism	Custom CSS	Frosted-glass UI aesthetic with backdrop-filter blur
@vitejs/plugin-basic-ssl	^2.3.0	Dev HTTPS required for getUserMedia on localhost

8. Features & User Experience

8.1 Drawing Features

- Real-time freehand stroke drawing via index finger pointing gesture.
- Exponential smoothing (factor 0.85) eliminates jitter from hand tremor.
- Customisable brush colour from 6 neon presets.
- Adjustable line width (1–50px) and glow intensity (0–50 sigma).
- Canvas 2D shadow rendering creates natural neon glow effect.
- Pinch-to-erase: radius-based stroke removal around fingertip cursor.
- Fist gesture clears the entire canvas instantly.
- Undo (pop from strokes[]) and redo (push back from redoStack[]).
- Export canvas to PNG with timestamp filename via download link.

8.2 Transform Features

- Two-finger MOVE gesture with inertia (0.92 decay velocity drift).
- Pinch SCALE with sensitivity × 8 multiplier; clamped 0.1× to 5.0×.
- Open-palm ROTATE with wrist angle tracking; snaps to 45° on release.

- Visual selection guides: blue crosshair (move), concentric rings + % label (scale), orange arc indicator (rotate).
- 4-frame gesture debounce prevents flicker between control gesture states.

8.3 UI / Settings

- Collapsible glassmorphism control panel (top-right, 280px wide).
- Toggle camera feed visibility without pausing tracking.
- Toggle gesture recognition on/off (useful for configuration).
- Animated gesture status HUD (bottom-centre) via Framer Motion.
- Built-in gesture guide modal with per-section cheat-sheet.
- Responsive canvas that resizes to full viewport on window resize.

9. Project Setup & Deployment

9.1 Prerequisites

- Node.js ≥ 18 and npm.
- Modern browser with WebRTC (getUserMedia) and WebGL support.
- HTTPS connection (required for camera access — provided by plugin-basic-ssl in dev).

9.2 Installation

```
# Clone / extract the project
cd "air draw/AirDrawer"

# Install dependencies
npm install

# Start development server (HTTPS)
npm run dev

# Build for production
npm run build
```

9.3 Environment Notes

- MediaPipe Hands model files are loaded at runtime from jsDelivr CDN — internet access required.
- Camera permission must be granted in the browser for hand tracking to function.
- Best performance on Chromium-based browsers (Chrome, Edge) with hardware acceleration enabled.

10. Limitations & Future Work

10.1 Current Limitations

- No backend or persistent storage — all strokes are lost on page refresh.
- Undo for erase operations is not implemented (simple removal, no state snapshot).
- Single canvas layer; no layer management or z-ordering of strokes.
- Camera feed is always mirrored; no option for non-mirrored mode.
- Gesture recognition can misfire under poor lighting or occlusion.

10.2 Suggested Enhancements

- IndexedDB or localStorage persistence for stroke sessions.
- Multi-layer canvas with layer reordering.
- WebSocket collaboration allowing multiple users to draw simultaneously.
- Voice commands as a secondary input channel.
- Improved erase-undo via immutable stroke snapshots.
- Custom colour picker beyond the 6 neon presets.
- Export to SVG for resolution-independent artwork.

11. Glossary

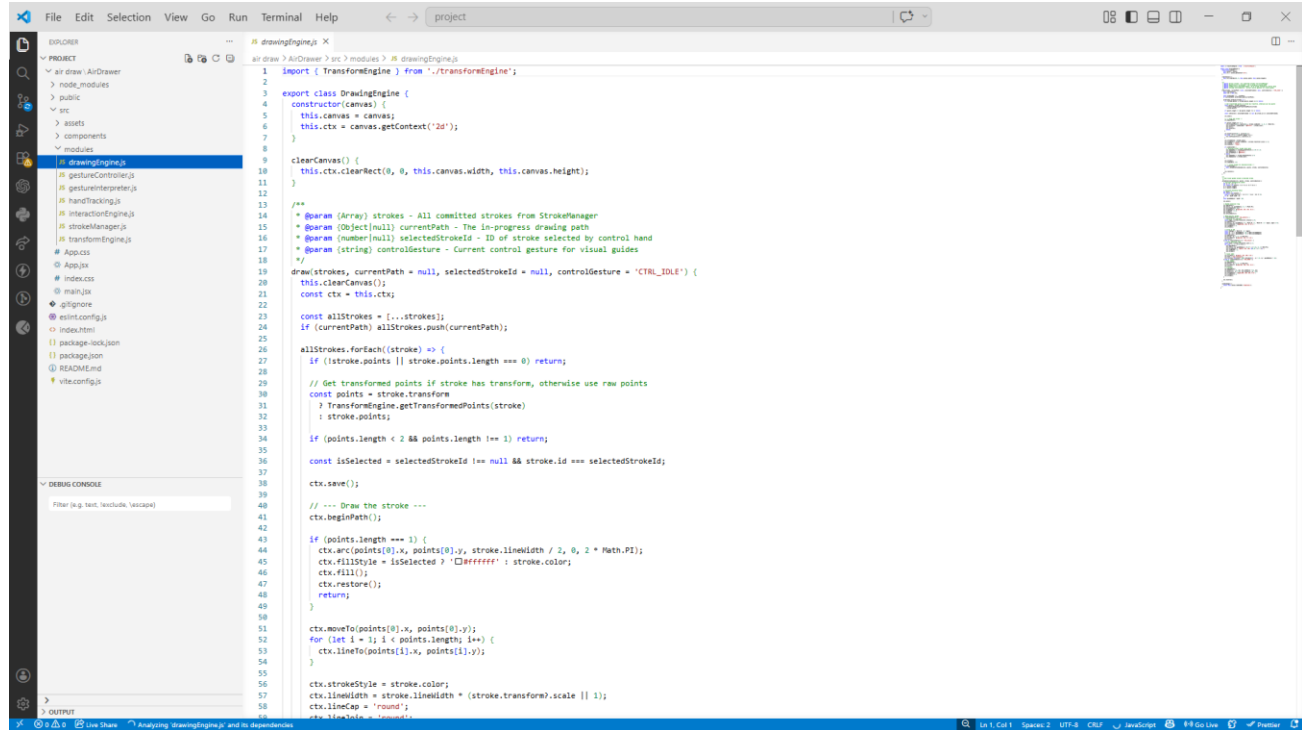
Term	Definition
Landmark	A 3D (x, y, z) point returned by MediaPipe for a specific hand joint.
Stroke	A single continuous pen-down path on the canvas, stored as an ordered array of points.
Transform	A set of (tx, ty, scale, rotation) values applied to a stroke at render time without altering the original points.
Pinch distance	The Euclidean distance (0–1 normalised) between the thumb tip and index tip landmarks.
Gesture debounce	Requiring N consecutive frames of a new gesture before switching, to prevent flickering.
Inertia	Continued motion after a gesture is released, decaying exponentially each frame.
Glassmorphism	A UI design style using frosted-glass panels with backdrop-filter blur, transparency, and subtle borders.
rAF loop	requestAnimationFrame loop — a browser API for smooth, display-sync animation callbacks (typically 60 FPS).

MediaPipe Hands

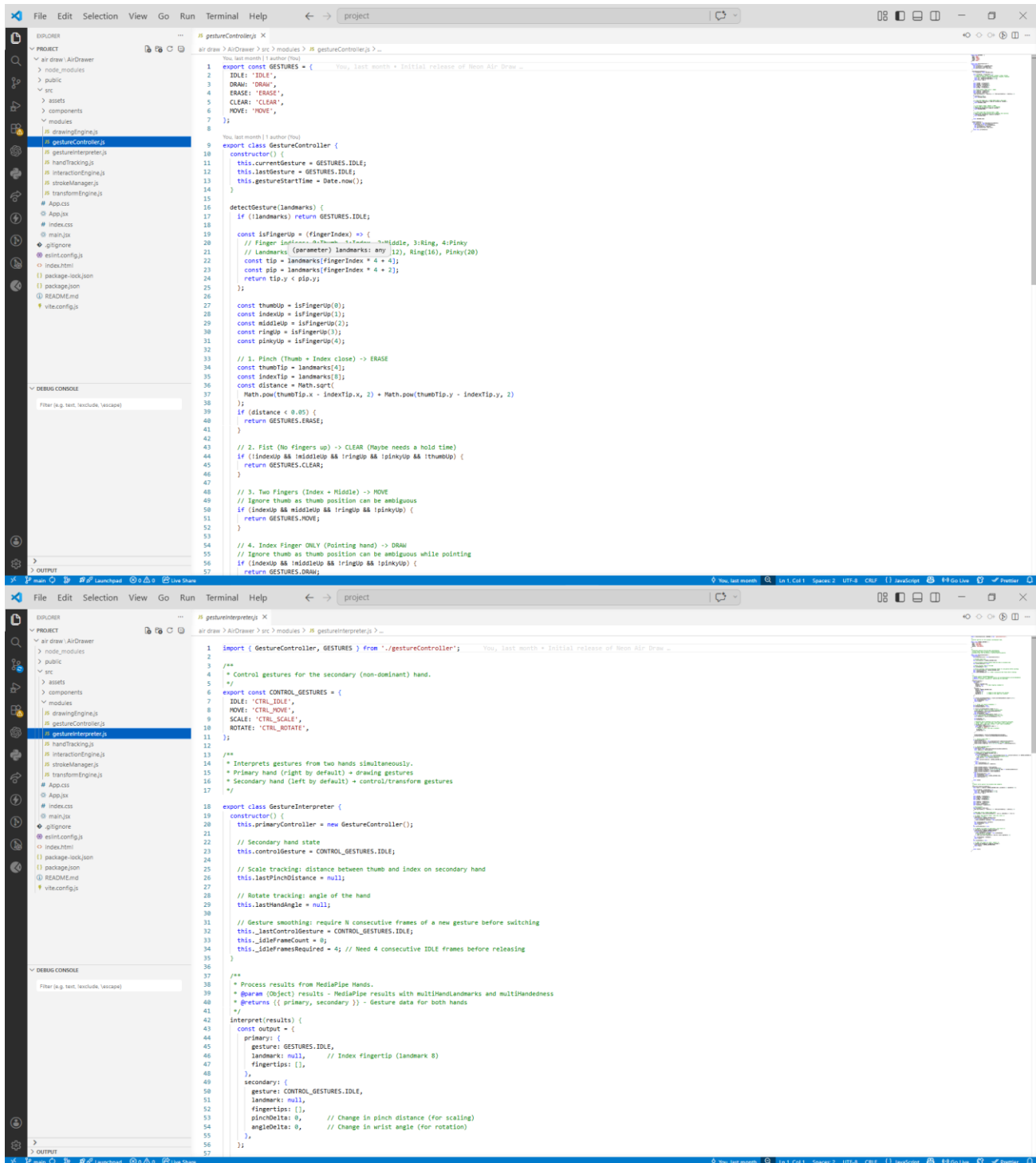
Google's ML framework for real-time hand and finger tracking from a standard webcam.

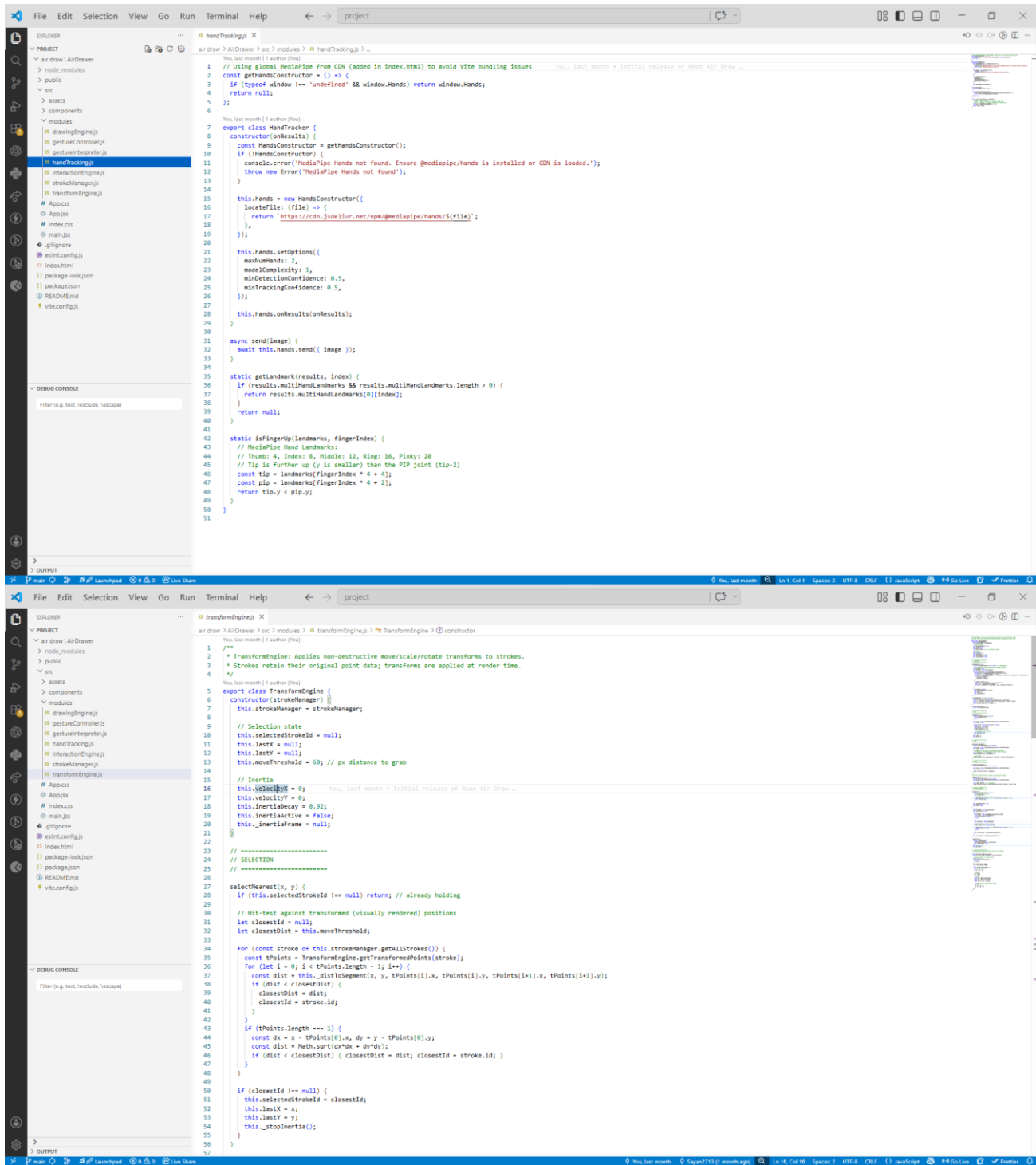
12. Code And Output

12.1 Code Screenshot

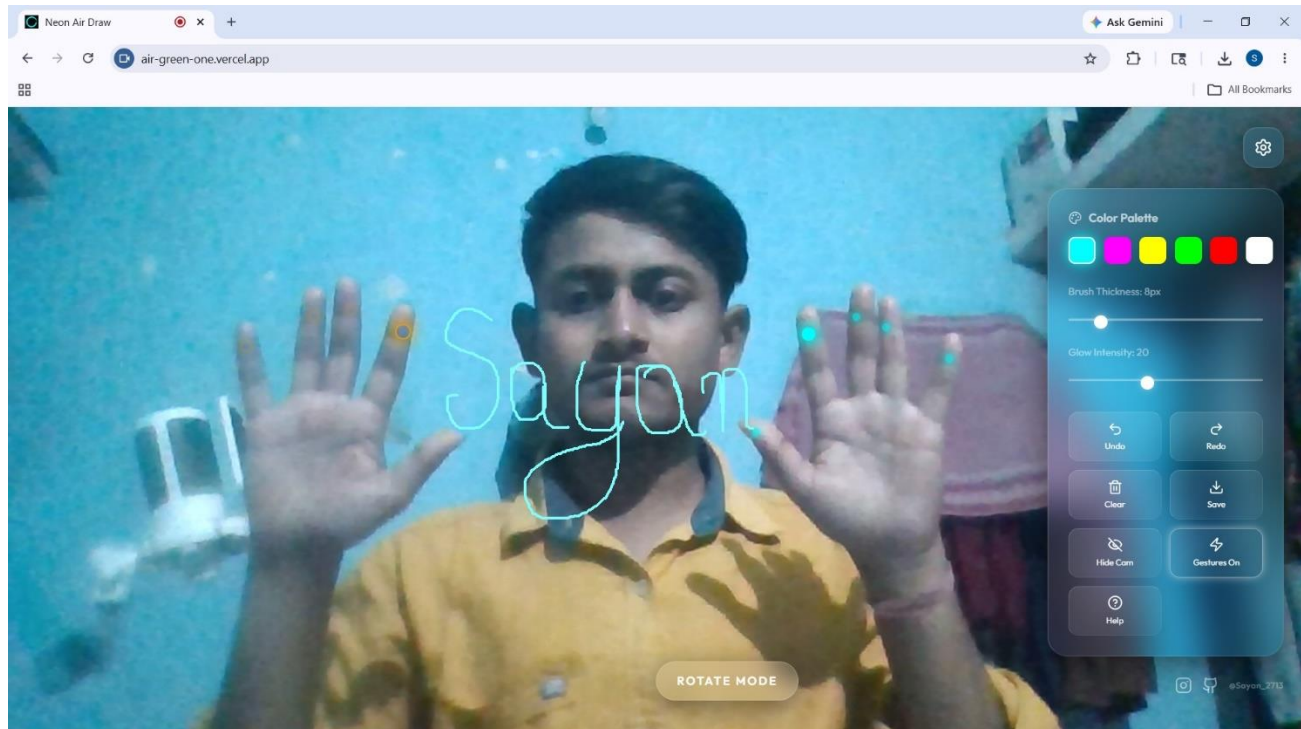


```
1 import { TransformEngine } from './transformEngine';
2
3 export class DrawingEngine {
4   constructor(canvas) {
5     this.canvas = canvas;
6     this.ctx = canvas.getContext('2d');
7   }
8
9   clearCanvas() {
10    this.ctx.clearRect(0, 0, this.canvas.width, this.canvas.height);
11  }
12
13  /**
14   * @param {Array} strokes - All committed strokes from StrokeManager
15   * @param {Object|null} currentPath - The in-progress drawing path
16   * @param {number|null} selectedStrokeId - ID of stroke selected by control hand
17   * @param {string} controlGesture - Current control gesture for visual guides
18   */
19  draw(strokes, currentPath = null, selectedStrokeId = null, controlGesture = 'CTRL_IDLE') {
20    this.clearCanvas();
21    const ctx = this.ctx;
22
23    const allStrokes = [...strokes];
24    if (currentPath) allStrokes.push(currentPath);
25
26    allStrokes.forEach((stroke) => {
27      if (!stroke.points || stroke.points.length === 0) return;
28
29      // Get transformed points if stroke has transform, otherwise use raw points
30      const points = stroke.transform
31        ? TransformEngine.getTransformedPoints(stroke)
32        : stroke.points;
33
34      if (points.length < 2 && points.length !== 1) return;
35
36      const isSelected = selectedStrokeId !== null && stroke.id === selectedStrokeId;
37
38      ctx.save();
39
40      // --- Draw the stroke ---
41      ctx.beginPath();
42
43      if (points.length === 1) {
44        ctx.arc(points[0].x, points[0].y, stroke.lineWidth / 2, 0, 2 * Math.PI);
45        ctx.fillStyle = isSelected ? 'black' : stroke.color;
46        ctx.fill();
47        ctx.restore();
48      }
49
50      ctx.moveTo(points[0].x, points[0].y);
51      for (let i = 1; i < points.length; i++) {
52        ctx.lineTo(points[i].x, points[i].y);
53      }
54
55      ctx.strokeStyle = stroke.color;
56      ctx.lineWidth = stroke.lineWidth * (stroke.transform ? scale : 1);
57      ctx.lineCap = 'round';
58      ctx.stroke();
59    });
60  }
```





12.2 Output



13. References

Books & Textbooks

1. Hartley, R. & Zisserman, A. (2004). *Multiple View Geometry in Computer Vision* (2nd ed.). Cambridge University Press.
2. Szeliski, R. (2022). *Computer Vision: Algorithms and Applications* (2nd ed.). Springer.

Research Papers

3. Zhang, F., et al. (2020). *MediaPipe Hands: On-device Real-time Hand Tracking*. arXiv:2006.10214.
4. Simon, T., et al. (2017). *Hand Keypoint Detection in Single Images using Multiview Bootstrapping*. CVPR 2017.
5. Cao, Z., et al. (2019). *OpenPose: Realtime Multi-Person 2D Pose Estimation*. IEEE TPAMI.

Official Documentation

6. Google LLC. (2023). *MediaPipe Hands Solution*. https://developers.google.com/mediapipe/solutions/vision/hand_landmarker
7. Meta Open Source. (2024). *React Documentation*. <https://react.dev>
8. Vite Contributors. (2024). *Vite Official Guide*. <https://vitejs.dev/guide>
9. Three.js Authors. (2024). *Three.js Documentation*. <https://threejs.org/docs>
10. Framer. (2024). *Framer Motion API Reference*. <https://www.framer.com/motion>

Web Standards

11. W3C. (2023). *HTML Canvas 2D Context Specification*. <https://html.spec.whatwg.org/multipage/canvas.html>
12. MDN Web Docs. (2024). *MediaDevices.getUserMedia()*. <https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia>

Related Work / Inspiration

13. Billinghurst, M., et al. (2015). *A Survey of Augmented Reality*. Foundations and Trends in HCI, 8(2–3), 73–272.
14. Bowman, D. & Hodges, L. (1997). *An Evaluation of Techniques for Grabbing and Manipulating Remote Objects in Immersive Virtual Environments*. ACM Symposium on Interactive 3D Graphics.